



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
NATIONAL COLLEGE OF ENGINEERING

A
MID-TERM PROJECT REPORT
ON
“EDUMENTX: A LOCATION BASED TUTOR DISCOVERY
PLATFORM FOR KATHMANDU VALLEY”

SUBMITTED BY:
APOORVA PAUDEL (NCE080BCT006)
ASIM POUDEL (NCE080BCT008)
DIBYA SHAKYA (NCE080BCT013)
SUHAN KHADKA (NCE080BCT043)

SUBMITTED TO:
DEPARTMENT OF ELECTRONICS & COMPUTER
ENGINEERING

LALITPUR, NEPAL
JULY 8, 2026

APPROVAL PAGE

National College of Engineering
Department of Electronics & Computer Engineering

The undersigned certify that they have read and recommend to the Department of Electronics & Computer Engineering for acceptance, a mid-term project report entitled “EdumentX: A Location Based Tutor Discovery Platform for Kathmandu Valley” submitted by the group listed on the title page in partial fulfillment of the requirements for the degree of Bachelor in Computer Engineering.

Project Supervisor

Head of Department

Date: _____

Date: _____

ACKNOWLEDGEMENT

[To be filled in before submission.]

ABSTRACT

[To be filled in before submission — typically 200–300 words covering problem, methodology, current progress, and expected outcome.]

LIST OF ABBREVIATIONS

API	Application Programming Interface
ASCILITE	Australasian Society for Computers in Learning in Tertiary Education
CBS	Central Bureau of Statistics (Nepal)
DFD	Data Flow Diagram
ESP	Electronically Stored Personal data
GBV	Geospatial Bounding Value
GPS	Global Positioning System
GUI	Graphical User Interface
HTTP(S)	Hypertext Transfer Protocol (Secure)
IDE	Integrated Development Environment
IOE	Institute of Engineering
KNN	K-Nearest Neighbors
LBS	Location-Based Service
LLM	Large Language Model
MFA	Multi-Factor Authentication
MOOC	Massive Open Online Course
NCE	National College of Engineering
NLP	Natural Language Processing
NoSQL	Not only SQL
OTP	One-Time Password
RAG	Retrieval-Augmented Generation
SEE	Secondary Education Examination (Grade 10, Nepal)
SDLC	Software Development Life Cycle
TU	Tribhuvan University
UAT	User Acceptance Testing
UI	User Interface
UX	User Experience

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1 INTRODUCTION

1.1 Background

Private supplementary tutoring, often called “shadow education,” is a globally significant phenomenon. UNESCO estimated that shadow education operates as a parallel, fee-based system of academic support that exists alongside mainstream schooling (?). In Asia, the scale of the sector is substantial: cross-national surveys report that a large majority of students in countries such as Nepal, India, Bangladesh, and Sri Lanka use some form of private tutoring outside of school hours (??).

In Nepal, the Central Bureau of Statistics (?) reports that private tuition is the dominant academic support channel outside of formal schooling, particularly in the Kathmandu Valley where population density, household disposable income, and competition for entrance examinations are highest. Amrit Thapa (?) documents how the rapid expansion of private schooling has been matched by a parallel rise in private tutoring, with the home-tuition segment — one-to-one or small- group sessions held at a student’s residence — remaining the preferred mode for SEE and +2 students.

Despite the scale of the market, three structural problems persist for students and parents searching for a home tutor. First, there is no reliable channel to discover independent tutors who teach in a specific neighborhood. Existing discovery mechanisms — social media groups, school notice boards, word of mouth, tuition agencies — do not provide spatial filtering, so a student in Lalitpur cannot easily tell which of fifty tutors listed in a Facebook group is within walking distance. Second, the qualifications of self-listed tutors are difficult to verify. Most listings are unverified text claims, with no standardized mechanism for a tutor to upload academic credentials and a demo video that a third party can review. Third, the discovery and screening process is manual, time-consuming, and biased toward tutors who already have social connections, leaving independent tutors (university students, retired teachers, part-time professionals) without a fair channel to reach nearby students.

EdumentX is a cross-platform mobile application that addresses these three

problems in a single integrated system. The application combines a map-based proximity search, an administrator-mediated verification system that grants a Blue Tick badge to tutors whose credentials have been reviewed, and a natural-language Retrieval-Augmented Generation (RAG) chatbot that returns personalized tutor recommendations grounded in live platform data.

1.2 Problem Statement

The mid-term report establishes the formal problem statements that motivate the EdumentX system:

1. Lack of location-aware tutor discovery. There is no Nepali platform that lets a student or parent search for home tutors within a configurable radius of their address and rank the result by proximity, qualification, price, and rating in a single view.
2. No trusted channel for tutor verification. Independent tutors have no standardized mechanism to publish academic credentials, identity documents, and a teaching demo video in a way that a third-party administrator can review and signal as trusted. Parents currently rely on informal claims and word of mouth.
3. No intelligent, conversational tutor recommendation. Existing manual filtering (subject, area, price, availability) requires the user to specify every parameter explicitly. There is no natural-language interface in the Nepali tutoring market that combines proximity, qualifications, availability, and budget into a single intelligent ranking powered by a language model grounded in live platform data.

1.3 Objectives

The objectives of the EdumentX project are:

1. To design and implement a cross-platform mobile application (Android and iOS) that supports map-based tutor discovery with proximity ranking for the Kathmandu Valley.
2. To build a tutor verification pipeline in which a tutor uploads academic credentials, identity documents, and a short teaching demo, and an administrator reviews the submission to grant a Blue Tick badge that is then surfaced in search results and the RAG chatbot’s response.

3. To implement a Retrieval-Augmented Generation chatbot that accepts free-text natural-language queries from students and parents, retrieves matching tutor profiles from live Firestore data using the Haversine formula and K-Nearest Neighbors ranking, and returns a ranked set of tutor recommendations in plain Nepali English.

1.4 Scope

The scope of the EdumentX project is defined as follows:

- Geographic scope: Kathmandu Valley (Kathmandu, Lalitpur, and Bhaktapur districts), with the system architected to support additional cities without major redesign.
- Service type: Home tuition — one-to-one and group sessions at a student's residence. Online-only tutoring is out of scope for this mid-term milestone.
- Academic levels: School and college students at SEE, +2, and undergraduate levels.
- Platform: Cross-platform mobile application for Android and iOS, built with React Native via Expo.
- Discovery methods: Map-based search with proximity ranking, and an AI chatbot for natural-language queries.
- Enrollment model: Long-term monthly enrollment plans of 1, 3, 6, or 12 months, managed through structured request-and-response flows.
- Verification: Blue Tick badge system for tutor credential verification by an administrator.
- Group learning: Support for group tuition batches created by tutors for shared, cost-effective learning.

2 LITERATURE REVIEW

2.1 Related Work

This section reviews recent academic research on location-based tutor matching systems and AI-powered educational platforms, and identifies the gaps that Edu-mentX aims to address.

A 2022 study presented at ASCILITE introduced MENTOR, an intelligent mobile peer tutoring application designed for both face-to-face and remote tutoring (?). The system focused on matching peer tutors with students based on subject expertise and availability. While it demonstrated the value of mobile platforms for tutoring, it did not include any location-based features such as map visualization or proximity ranking — the matching was based solely on academic parameters without considering geographic distance.

A 2026 study published in the Journal of Emerging Technologies and Innovative Research described a Flutter-based mobile application for finding local tutors in India (?). The system used Google Maps API for proximity-based tutor search and Firebase as the backend database. Students could filter tutors by subject, availability, and location, and the application calculated distances between users using the Haversine formula. The researchers noted, however, that AI-based recommendations were left as future scope — the entire matching process relied on manual filtering by the user with no intelligent ranking or conversational search capability.

In 2025, researchers from Malaysia developed iTeachU, a system that combined AI with geolocation for tutor matching (?). The platform used content-based filtering along with the K-Nearest Neighbors algorithm, incorporating Haversine distance calculations to rank tutors by both preference match and physical proximity. The system demonstrated effective ranking within the Melaka region, but its AI component was entirely rule-based — users could not interact with the system using natural language, and instead had to select from predefined filters and dropdown menus.

All of these systems represent important steps toward technology-driven tutor discovery, but several gaps remain when considering the Nepali context. No exist-

ing research or platform provides geospatial visualization of tutors on interactive maps with service-area coverage for Nepal. City-level dropdown menus are not useful in dense urban areas like Kathmandu, where a tutor two kilometers away is materially different from one ten kilometers away. Independent part-time tutors — university students and retired teachers — lack a proper digital channel to reach nearby students because existing systems tend to favor agency-managed tuition centers. None of the reviewed systems offer a natural-language interface where users can type what they need, and none combine proximity, qualifications, availability, and budget into a single intelligent ranking powered by a large language model. EdumentX addresses these gaps by combining map-based proximity search, a structured verification system with Blue Tick badges, and a RAG-based conversational interface for natural-language tutor discovery in the Kathmandu Valley.

2.2 Related Theory

2.2.1 Location-Based Services

Location-Based Services deliver context-aware services using a device’s real-time geographic position obtained through GPS, cellular triangulation, or Wi-Fi. In EdumentX, location-based services power the proximity-based tutor discovery and map visualization features. Tutor coordinates are stored as Firestore GeoPoint fields, which allow geospatial queries to be executed directly from the client application.

2.2.2 Haversine Distance Formula

The Haversine formula computes the great-circle distance between two points on Earth’s surface, accounting for the planet’s curvature. EdumentX uses this formula to rank tutors by their true distance from the student. The computation proceeds as follows:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (2.1)$$

$$c = 2 \arctan 2(\sqrt{a}, \sqrt{1-a}), \quad d = R \cdot c \quad (2.2)$$

where ϕ_1, ϕ_2 are latitudes in radians, λ_1, λ_2 are longitudes in radians, and $\Delta\phi$ and $\Delta\lambda$ are the coordinate differences. $R \approx 6,371$ km is Earth’s mean radius and d is the output distance. For the 1–20 km range typical in the Kathmandu Valley, this formula provides meter-level accuracy.

2.2.3 Geofencing and Radius-Based Filtering

Before applying the Haversine formula, EdumentX uses a rectangular bounding-box pre-filter to narrow down the candidate set and reduce computation (?). For a given search radius r and student position (ϕ_s, λ_s) :

$$\Delta\phi_{\max} = \frac{r}{R} \cdot \frac{180}{\pi}, \quad \Delta\lambda_{\max} = \frac{r}{R \cdot \cos(\phi_s)} \cdot \frac{180}{\pi} \quad (2.3)$$

Only tutors whose coordinates fall inside this bounding box proceed to the exact Haversine distance check, which keeps query response times low.

2.2.4 K-Nearest Neighbors Algorithm

KNN is a non-parametric algorithm used in EdumentX to rank tutors by overall suitability. Each tutor is represented as a feature vector $\mathbf{t} \in \mathbb{R}^5$:

$$\mathbf{t} = [d, r, q, e, \bar{v}] \quad (2.4)$$

where d is the Haversine distance, r is the monthly rate, q is the qualification score, e is years of experience, and $\bar{v}$ is the average rating. All features are scaled to the range $[0, 1]$ using min-max normalization: $x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$. The similarity between a student query vector $\mathbf{s}$ and a tutor vector $\mathbf{t}$ is measured by weighted Euclidean distance:

$$D(\mathbf{s}, \mathbf{t}) = \sqrt{\sum_{j=1}^n w_j \cdot (s_j - t_j)^2}, \quad \text{Top-}k = \arg \min_{\mathbf{t} \in \mathcal{T}} D(\mathbf{s}, \mathbf{t}) \quad (2.5)$$

The default value of k is 5. The feature weights are tuned for the home-tuition selection context and are summarized in Table ??.

Table 2.1: KNN Feature Weights for Tutor Recommendation

Feature	Symbol	Weight w_j
Geographic Distance	d	0.35
Monthly Rate	r	0.25
Qualification Score	q	0.20
Average Rating	$\bar{v}$	0.15
Experience (years)	e	0.05

2.2.5 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with text generation to produce responses grounded in actual data rather than relying solely

on a language model’s training corpus. The original RAG framework was introduced by Lewis et al. (?), who showed that combining a pre-trained retriever with a sequence-to-sequence generator significantly improves accuracy on knowledge-intensive tasks. In EdumentX, RAG is used to retrieve matching tutor profiles from Firestore based on the user’s query and inject them as context into the language model’s prompt, ensuring that recommendations are based on real, live platform data. A recent survey on RAG chatbots in education (?) confirms that this approach reduces hallucination and improves the relevance of generated responses in educational contexts.

2.2.6 React Native and Expo

React Native enables building native iOS and Android applications from a single JavaScript and React codebase. Expo adds a managed toolchain for accessing device features such as location, camera, and push notifications, and supports over-the-air updates that shorten the development and release cycle. EdumentX uses Expo SDK 54 with the New Architecture (Fabric/TurboModules) enabled.

2.2.7 Firebase Platform

Firebase is Google’s Backend-as-a-Service suite and forms the primary backend for EdumentX. Four services are integrated: Authentication for email/password and Google identity-provider sign-in; Firestore for real-time NoSQL storage with native GeoPoint fields and Security Rules; Supabase Storage for verification documents and demo videos; and the Firebase Admin SDK (Node.js, used out-of-band) for bootstrapping the administrator role. Note that EdumentX does not depend on Cloud Functions: the client computes Haversine distance, KNN ranking, and the retrieval step for the RAG pipeline directly. This keeps the project within the free Spark tier and avoids the paid Blaze plan that the project team cannot sign up for.

2.2.8 OpenStreetMap and Open Geocoders

To avoid the paid Google Maps Platform API, EdumentX uses OpenStreetMap raster tiles rendered through react-native-maps’ `<UrlTile>` component (no API key) and the keyless Nominatim service for forward geocoding. Distance and KNN matching are computed client-side, which is feasible at the Kathmandu-Valley scale (target database size is in the low thousands of tutors).

3 METHODOLOGY

3.1 Software Development Model

The EdumentX project follows the Agile Iterative-Incremental development model. The system is built through short development cycles called sprints, where each sprint delivers a working piece of the application. After each sprint, the output is reviewed, tested, and improved before moving to the next cycle. This model suits EdumentX because the project has distinct modules — authentication, profile onboarding, map search, AI chatbot, verification, enrollment — that can be developed independently. The 14-week academic timeline is divided into seven two-week sprints with clear deliverables at each stage. Figure ?? shows the phases of the iterative-incremental model.

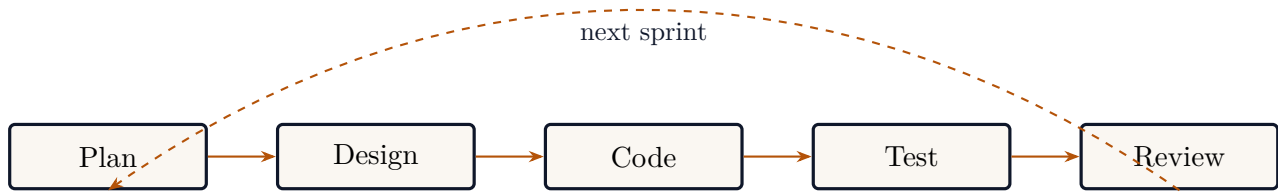


Figure 3.1: Phases of the Iterative-Incremental Model

3.2 System Architecture

EdumentX is structured as a thin React Native client that talks directly to a Firebase backend. The architecture has three layers: Client Side, Serverless Backend, and External Services (Figure ??).

The Client Side is the React Native mobile application built with Expo. It serves as the single entry point for all users and communicates with the backend using the native Firebase SDK (`@react-native-firebase/*`).

The Serverless Backend runs entirely on Firebase with three services. Firebase Authentication handles user registration and email/password plus Google sign-in. Cloud Firestore stores all structured data including user profiles, tutor profiles, enrollment requests, and reviews, with tutor locations stored as GeoPoint fields and

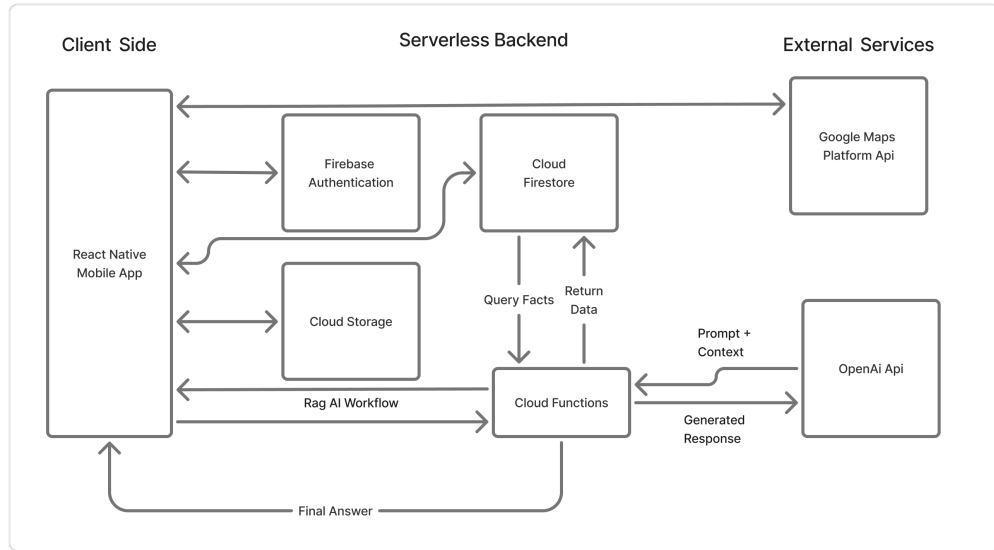


Figure 3.2: System Architecture of EdumentX

protected by Security Rules. Supabase Storage holds tutor verification documents and demo videos.

The External Services layer includes OpenStreetMap tiles (rendered via `<UrlTile>`) for map rendering, Nominatim for keyless forward geocoding, and a free-tier LLM endpoint (Groq or HuggingFace Serverless Inference) for the RAG chatbot. Importantly, EdumentX does not depend on Cloud Functions, Google Maps SDK, or any paid LLM — every external call is either keyless or fits within a free quota.

3.3 Use Case Diagram

Figure ?? shows the use case diagram with three actors: Students/Parents, Tutors, and Administrators. Students and parents can register, turn on location, search for tutors using filters or the AI chatbot, view tutor profiles, send enrollment requests, and rate tutors after completing an enrollment. Tutors can register, set their service area, upload documents for verification, manage their profile and availability, respond to enrollment requests, create group batches, and manage student capacity. Administrators can log in to the admin panel, verify tutor credentials, manage user accounts, and grant Blue Tick badges.

3.4 Data Flow Diagram

The Level 0 Data Flow Diagram (Figure ??) shows EdumentX as a single process interacting with Students/Parents, Tutors, and the Firestore database. Users interact with the system by sending requests and receiving responses; the system

processes these requests and stores or retrieves data from Firestore.

The Level 1 Data Flow Diagram (Figure ??) expands the EdumentX process into its sub-processes: authentication, tutor search, AI chatbot recommendation, enrollment management, tutor verification, and rating handling.

3.5 RAG Chatbot Workflow

The EdumentX RAG chatbot is implemented client-side to stay within the free tier. Figure ?? illustrates the six-step workflow. Step 1: the student types a natural language query such as “I need a science tutor near Lalitpur for 3 days a week under Rs 5,000.” The full conversation history is included so the chatbot can handle follow-up questions. Step 2: the client extracts structured parameters from the query — subject, location, budget, days per week, plan duration — using a keyless LLM endpoint. Step 3: the client queries the Firestore tutors collection, applies the bounding-box pre-filter (Section ??) followed by exact Haversine distance, and applies budget and subject filters. Step 4: the top ten matching tutor profiles are formatted and injected into the LLM’s prompt as context data — this is the retrieval-augmented step. Step 5: the language model generates a response recommending three to five tutors with name, rate, distance, rating, and verification status. Step 6: the response is rendered as a chat message in the app. Identical queries are cached for 15 minutes to keep the free-tier token budget healthy.

3.6 Implementation Details

3.6.1 Frontend Development

The frontend is built using React Native with Expo SDK 54 and the New Architecture enabled. The styling system is NativeWind 4.2.x (a Tailwind compiler for React Native) with a custom “Night & Sand” premium design palette (slate-night background, sand surface, amber accent) defined in `tailwind.config.js`. The codebase uses standard React Native primitives (View, Text, Pressable, TextInput, ScrollView) and avoids inline style objects. State is managed with Zustand for global session state and React context for screen-local state. Navigation uses expo-router on top of React Navigation.

Key screens implemented at the mid-term milestone include authentication (email + password, Google Sign-In), role selection, student and tutor profile onboarding, student and tutor home dashboards, profile parity screens (with avatar upload), and an admin surface (verification queue, statistics, user management, admin profile setup).

3.6.2 Backend Development

The backend uses a pure Firebase stack with no custom server. Firebase Authentication handles email/password and Google sign-in and session management. Firestore stores all application data with tutor locations as GeoPoint fields, and Security Rules control read/write access per role. Supabase Storage (1 GB free, no card) holds verification documents and demo videos. Composite indexes on subject, location, and rate fields enable efficient multi-criteria searches. The Firebase Admin SDK is used out-of-band from a local Node.js script (`scripts/seedAdmin.ts`) to grant the administrator role to a fixed allow-list of email addresses, since the `admins` collection is locked to Admin-SDK writes only.

3.6.3 Testing Strategy

Unit testing is performed with Jest for individual React Native components and pure functions (Haversine, KNN, RAG cache). Integration testing covers Firebase Authentication flows, Firestore Security Rules, and the upload-to-Supabase pipeline. User acceptance testing is being conducted with a small group of student and tutor test accounts. Performance testing focuses on cold-start time of the Expo dev client, map-tile render latency, and the end-to-end RAG query latency at typical Kathmandu-Valley data sizes (tens to low thousands of tutors).

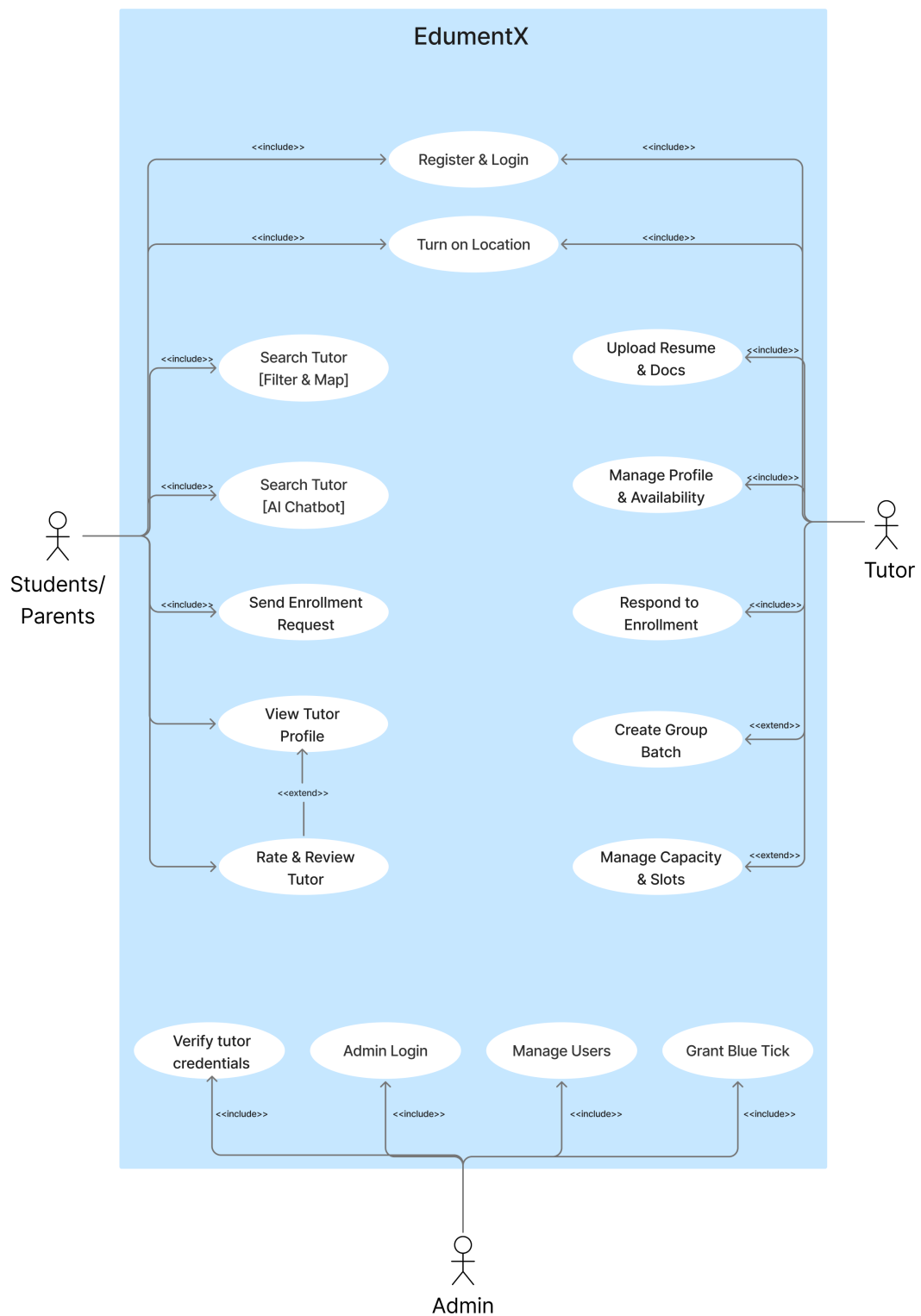


Figure 3.3: Use Case Diagram of EdumentX

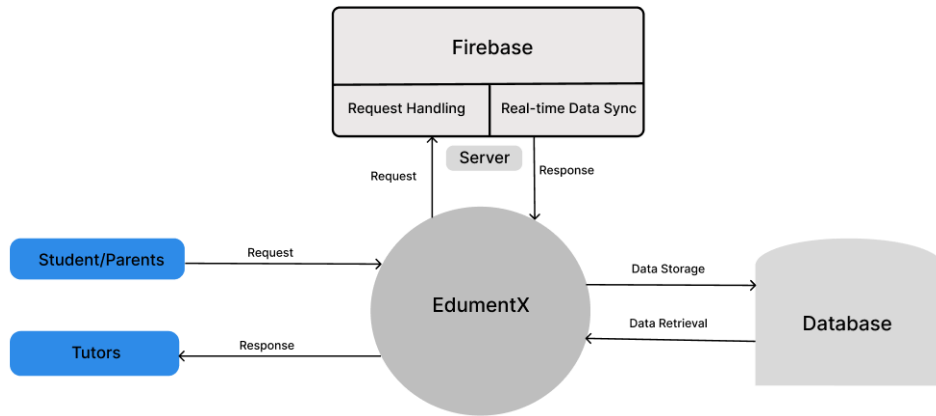


Figure 3.4: Level 0 Data Flow Diagram

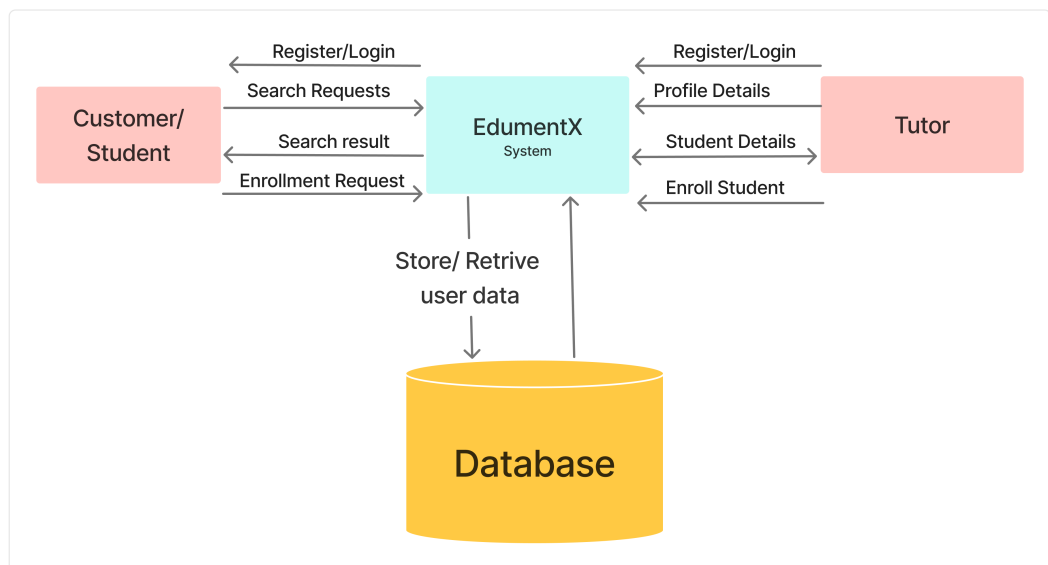


Figure 3.5: Level 1 Data Flow Diagram

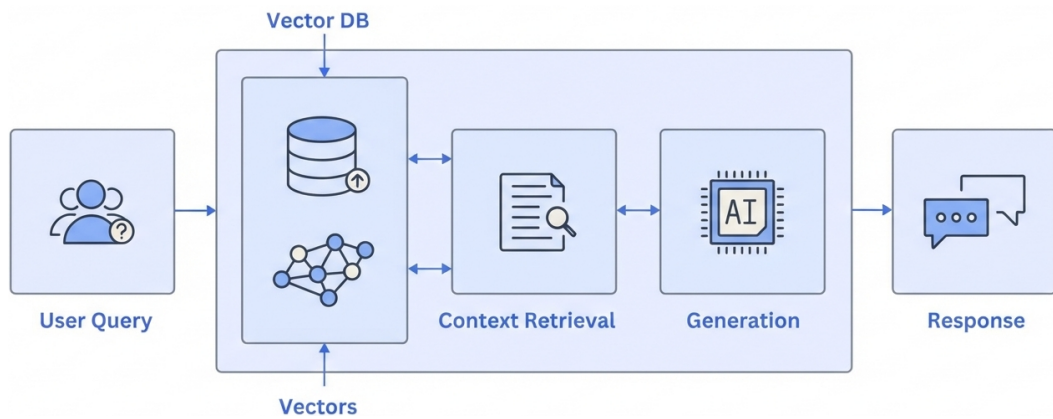


Figure 3.6: RAG-Based AI Chatbot Workflow

4 TASK ACCOMPLISHED

This chapter summarizes the work completed as of the mid-term defense. EdumentX is being built as a sequence of numbered stages; each stage is independently demonstrable. Stages 0 through B are complete and ready for committee review. Stages C onward are listed in Chapter ??.

4.1 Stage 0 — Project Setup and Documentation Scaffolding

- Repository initialized with Expo SDK 54, TypeScript, React Native, and the expo-router file-based router.
- Documentation/ folder hierarchy established (00-Overview, 01-Architecture, 02-Design-System, 03-Implementation-Guides, 04-Firebase, 05-Build-and-Deploy, 06-Prompts, 07-Design-Context, 97-Educational-Contents, 98-Reference-BasoBas, 99-Archive).
- LOG.md created and used as the team-wide change log. EdumentX_Master_Context_Sta established as the single source-of-truth status document.

4.2 Stage 1 — Design System: “Night & Sand” Premium Palette

- Design tokens registered in tailwind.config.js covering surface, text, border, and brand colors (night, sand, amber, etc.).
- constants/colors.ts provides raw hex values for the narrow set of SVG illustrations that need them.
- Shared primitives (Button, Card, Screen, Header) extracted to components/.

4.3 Stage 1.5 — NativeWind Migration (Tamagui Removal)

- Removed all @tamagui/* packages and the tamagui.config.ts file.
- Migrated the entire codebase from Tamagui styled components to standard React Native primitives with NativeWind className props.

- Replaced the Tamagui theme provider with a plain NativeWind setup driven by the `tailwind.config.js` tokens.

4.4 Stage 2 — Native Firebase Auth (RNFirestore Pivot)

- Adopted the native `@react-native-firebase/*` SDK packages (modular API: `getApp`, `getAuth`, `getFirestore`, `createUserWithEmailAndPassword`, `signInWithCredential`).
- `google-services.json` shipped at the native build time; no `process.env` Firebase config.
- Implemented two sign-in methods on `screens/auth/EmailSignUp.tsx`: Email + Password (`createUserWithEmailAndPassword` + `sendEmailVerification`) and Google Sign-In (`@react-native-google-signin/google-signin` → `signInWithCredential`).
- Auth Source-of-Truth routing lives in `app/_layout.tsx` as a 5-step decision tree: `unauthenticated` → `/email-signup`; `authenticated but unverified and password-provider` → the “check your inbox” pending panel; `verified but role-less` → `/role-selection`; `verified with role` → matching dashboard.
- Resolved the “Amnesia Login Loop” bug (existing user was being sent to `/role-selection` on every sign-in) by reading `users/{uid}.role` inside the `onAuthStateChanged` callback and writing it to the Zustand store before the redirect effect runs.

4.5 Stage 2b — Onboarding and Profile Subcollection Writes

- `screens/auth/RoleSelection.tsx` writes role to `users/{uid}` and routes to `/profile-student` or `/profile-tutor`.
- `screens/auth/StudentProfileScreen.tsx` and `screens/auth/TutorProfileScreen.tsx` use `writeBatch` to atomically write both the parent `users/{uid}` doc and the `users/{uid}/<role>Profile/default` subcollection doc, with `photoUrl` included as an optional field once `AvatarUploader` succeeds.
- Onboarding collects: name, phone, location (city-level Nominatim geocode), and role-specific fields (student: grade and subjects; tutor: headline, bio, monthly rate, subjects, years of experience).
- `LocationField` enforces `MIN_CITY_LENGTH = 3` for the city-field validation rule.

4.6 Stage 3 — Dashboards and Live Firestore Reads

- `screens/student/student_home.tsx` subscribes to the student's `users/{uid}` and `users/{uid}/studentProfile/default` docs via `onSnapshot` and shows the real `fullName` and `locationLabel`.
- `screens/tutor/tutor_home.tsx` does the same for tutors and surfaces the verified flag.
- `MOCK_TUTORS` list in `student_home.tsx` is a placeholder for the tutors collection query, which will be wired in Stage C.
- Tutor metric values (rating, reviews, response rate, monthly earnings) are still mock data in `tutor_home.tsx`; the live aggregation is queued for Stage C.

4.7 Stage A — Auth UX Polish, Bottom Nav, and Parity Foundations

- “Sign up” / “Log in” toggle on `EmailSignUp.tsx` with a “check your inbox” panel after signup. The “I’ve verified — continue” button is the only place the app calls `auth.currentUser.reload()`; without it, the cached `User.emailVerified` flag stays stale and the layout guard refuses to advance.
- Fixed the bottom-nav routing so that the tutor “Profile” tab now lands on `EditTutorProfile` instead of looping back to the dashboard.
- Real `fullName` and `locationLabel`/verified-flag data wired into the dashboard heroes and the profile view.
- CTA buttons constrained to a max width so they do not stretch across the full screen on tablets.
- Admin surface scaffolded: `AdminHome`, `Statistics`, `VerificationQueue`, `UserManagement`, and `AdminProfileSetup`. Administrator access is granted out-of-band by writing `admins/{uid}` from a local `firebase-admin` seed script (`scripts/seedAdmin.ts`) because the `admins` collection is locked to `Admin-SDK` writes only.

4.8 Stage A.5 — Tutor Profile Parity, Avatar Persistence, and Logout

- `screens/tutor/edit_profile.tsx` rewritten as a parity screen modeled on `StudentProfile.tsx`: slate hero, identity card with avatar bubble, name, email

and a Change-photo pill, two EditableField rows (Headline + About me), locked email row, three MenuRows (Availability, My batches, Payouts), Help & support MenuRow, destructive Log out button, and a version footer.

- MenuRow extracted from StudentProfile.tsx into components/forms/MenuRow.tsx so both screens import the same primitive.
- photoUrl persistence added in three places: TutorProfileScreen.tsx and StudentProfileScreen.tsx include photoUrl: avatarUri in the onboarding write-Batch payload; the new edit_profile.tsx parity screen and the StudentProfile.tsx screen both auto-persist photoUrl to Firestore via a useEffect on local-state change so the avatar survives a reload.
- Tutor profile tab now has a working destructive Log out button. The previously unused handleSignOut in tutor_home.tsx was removed (clearing an eslint warning).
- Upload pipeline replaces the “pickImage → Coming soon” alert with a real uploadAvatar call to Supabase Storage’s public-avatars/{uid}.jpg bucket with a cache-busting query parameter.

4.9 Stage B — 3D Onboarding Polish

- Onboarding slide 1 (DiscoverScene3D) and slide 2 (AiOrb3D) re-rendered with the new “Night & Sand” palette. The 3D renderer is loaded from react-three-fiber/native only (never the default react-three-fiber entry, which pulls DOM-only Web dependencies into the bundle).
- Metro config extended to support react-three-fiber/native and the asset pipeline for the 3D scenes.

4.10 Verification Summary

The mid-term codebase passes the following gates:

- npm run typecheck (TypeScript strict mode, tsc --noEmit) returns 0 errors.
- npm run lint returns 0 errors and a small, audited set of warnings (down from baseline during the parity rewrite). All warnings are pre-existing and have been triaged.
- firebase deploy --only firestore:rules applies the locked-down admins collection rules so the seed script can grant admin access.

- Manual end-to-end test on Android: sign up as student, complete onboarding, see real name in dashboard, change avatar, log out, sign back in as tutor, complete tutor onboarding, see real headline on profile tab, log out, sign in as admin (after running `npm run seed:admin`), complete admin profile setup, land on the admin dashboard.

5 TASKS REMAINING

[This chapter is a placeholder. The team will fill it in incrementally as work progresses toward the minor mid-term defense. The expected structure is: a short paragraph for each remaining stage, listing the deliverables, the rationale for why the stage is sequenced where it is, and the target sprint.]

5.1 Stage C — Live Tutor Query and KNN Ranking

[To be filled in.]

5.2 Stage D — RAG Chatbot End-to-End

[To be filled in.]

5.3 Stage E — Map Search and Service-Area Visualization

[To be filled in.]

5.4 Stage F — Verification Queue and Blue Tick Pipeline

[To be filled in.]

5.5 Stage G — Enrollment Request and Response Flow

[To be filled in.]

5.6 Stage H — Rating and Review System

[To be filled in.]

5.7 Stage I — Admin Statistics, User Management, and Polish

[To be filled in.]

5.8 Stage J — Final Hardening, Performance Pass, and Demo

[To be filled in.]

REFERENCES

BIBLIOGRAPHY

- M. Bray, *The Shadow Education System: Private Tutoring and Its Implications for Planners*. Paris: UNESCO, International Institute for Educational Planning, 2007.
- M. Bray and C. Lykins, *Shadow Education: Private Supplementary Tutoring and Its Implications for Policy Makers in Asia*. CERC Monograph Series in Comparative and International Education and Development, Manila: Asian Development Bank, 2012.
- Y. C. Kim and J. H. Jung, “Shadow education as worldwide curriculum studies,” *Curriculum Studies Worldwide*, vol. 1, pp. 1–15, Apr. 2019.
- Central Bureau of Statistics, “National population and housing census 2021: National report.” Government of Nepal, National Planning Commission, 2023. Census report.
- A. Thapa, “Does private school competition improve public school performance? the case of nepal,” *International Journal of Educational Development*, vol. 33, no. 4, pp. 358–366, 2013.
- S.-H. Chung and S. C. Tan, “MENTOR – intelligent mobile online peer tutoring application for face-to-face and remote peer tutoring,” in *ASCILITE Publications*, pp. 386–391, Sept. 2022.
- R. K. Sharma, A. Patel, and M. Singh, “Local tutor finder application using flutter,” *Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 13, pp. 267–275, Feb. 2026.
- N. Shari, F. Ahmad, and R. Hassan, “iTeachU: AI-driven tutor matching with geolocation integration,” *International Journal of Research in Science and Innovation (IJRSI)*, vol. 12, pp. 145–158, Mar. 2025.
- Y. Shevchenko and U.-D. Reips, “Geofencing in location-based behavioural research: Methodology, challenges, and implementation,” *Behavior Research Methods*, vol. 56, Aug. 2023.

- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks.” arXiv preprint arXiv:2005.11401, 2021.
- J. Swacha and M. Gracel, “Retrieval-augmented generation (RAG) chatbots for education: A survey of applications,” Applied Sciences, vol. 15, no. 8, p. 4234, 2025.